

INSURELLM • KNOWLEDGE HUB

InsureLLM

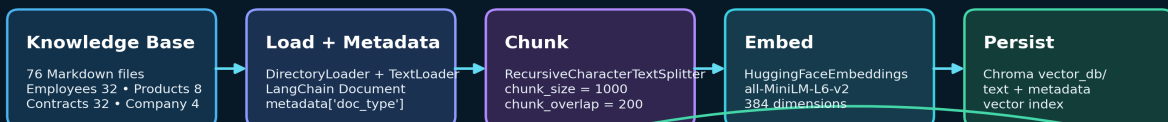
Functional, Technical & Architecture Reference

Developers • QA • Architects • Engineering Managers

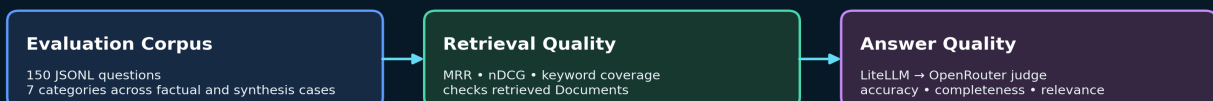
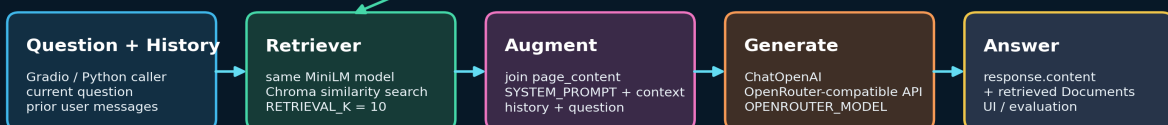
InsureLLM Code-Grounded RAG Architecture

Derived from the supplied notebooks, implementation/answer.py, evaluation code and repository structure

OFFLINE INDEXING



ONLINE RAG



Purpose. A source-grounded professional reference for the InsureLLM codebase: deep functional coverage, code-first developer explanations, QA/evaluation guidance, multiple architecture views, security/operations, management context, and the day5 advanced-RAG variant at the end.

Table of Contents

Table of Contents	2
1. Functional Overview — What InsureLLM Does	4
1.1 Business purpose	4
1.2 Primary capabilities	4
1.3 Knowledge scope	4
1.4 End-user flow	4
1.5 Current functional boundaries	4
2. Functional Requirements & Quality Expectations	6
2.1 Quality dimensions	6
3. Architecture — Canonical & Architect Views	7
3.1 Exact user-supplied `insurllm architecture.png`	7
3.2 Code-grounded canonical architecture	8
3.3 System context	8
3.4 Component architecture	9
3.5 Runtime sequence	9
3.6 Deployment and production evolution	10
3.7 Architect interpretation	10
4. Technology Stack & Libraries	11
4.1 LangChain's role	11
5. Developer Deep Dive — Code Snippet First, Explanation Second	12
5.1 Core imports	12
5.2 Embedding model	12
5.3 Build / rebuild Chroma	12
5.4 Retriever and LLM setup	13
5.5 Inspect embeddings and metadata	13
5.6 Custom `Result` model	14
5.7 Advanced `Chunk` and `Chunks` models	14
5.8 Reopen the persistent vector store	15
5.9 `fetch_context()`	15
5.10 Conversation-aware retrieval query	15
5.11 Complete RAG answer function	15
5.12 Embedding visualization	17
5.13 Restarting the Jupyter kernel	17

6. QA & Evaluation	18
6.1 Evaluation corpus	18
6.2 Retrieval metrics	18
6.3 Answer metrics	18
6.4 QA scenario matrix	18
6.5 Regression gate	18
7. Architect Deep Dive	19
7.1 Architectural quality attributes	19
7.2 Recommended index identity	19
7.3 Production service decomposition	19
7.4 Trust boundaries	19
8. Security, Privacy & Operations	20
8.1 Secret handling	20
8.2 RAG-specific risk areas	20
8.3 Production telemetry	20
9. Engineering Manager / Product View	21
9.1 Business value	21
9.2 Roadmap priorities	21
10. Advanced RAG — `day5.ipynb` (Placed Last Intentionally)	22
10.1 Configuration	22
10.2 Native document loading	22
10.3 LLM semantic chunking	22
10.4 Advanced embedding and native Chroma	23
10.5 Query rewrite	23
10.6 Candidate retrieval	23
10.7 LLM reranking	23
10.8 Final advanced answer function	25
10.9 Baseline vs advanced	25
11. Onboarding Checklist & Mental Models	26
11.1 New developer / QA sequence	26
11.2 Baseline mental model	26
11.3 Advanced mental model	26
11.4 Definition of done for a RAG change	26

1. Functional Overview – What InsureLLM Does

1.1 Business purpose

InsureLLM is an insurance-domain knowledge assistant that gives users conversational access to private company knowledge. Instead of depending only on an LLM's pretrained knowledge, the application retrieves relevant company passages and injects them into the model prompt before an answer is generated.

The functional objective is therefore broader than chat: find evidence, supply evidence, generate a useful answer, and measure whether both retrieval and generation are working well.

1.2 Primary capabilities

- Knowledge ingestion: load company, employee, product and contract Markdown files.
- Metadata enrichment: preserve source and `doc\_type` with each Document.
- Chunking: split documents into overlapping retrieval units.
- Semantic indexing: create embeddings and persist them in Chroma.
- Natural-language retrieval: search for semantically relevant chunks rather than exact words only.
- Conversational Q&A: use prior user turns to improve retrieval for follow-up questions.
- Prompt grounding: include retrieved evidence in the system prompt.
- Chat generation: invoke a model through OpenRouter using `ChatOpenAI`.
- Interactive demo: expose the flow through Gradio.
- Evaluation: measure retrieval and answer quality against a 150-question corpus.

1.3 Knowledge scope

Domain	Files	Functional scope
Employees	32	People, roles, backgrounds and employee facts.
Products	8	Insurance/product information.
Contracts	32	Contract facts and terms.
Company	4	Company-level information.
Total	76	All Markdown source documents.

1.4 End-user flow

- User asks a company-specific question.
- Application builds a retrieval query using the current question and relevant earlier user turns.
- Query is embedded using the same embedding model used for the index.
- Chroma returns the highest-ranked Documents.
- Retrieved `page\_content` becomes LLM context.
- Conversation history and current question are sent with the context to the model.
- Answer text is returned; retrieved Documents remain available for diagnostics/evaluation.

1.5 Current functional boundaries

- No end-user citations are rendered in the baseline UI.
- No production authentication/authorization layer.
- No metadata filter, hybrid search, reranker or relevance threshold in baseline retrieval.
- Index creation remains notebook-driven.
- Current local Chroma design is a prototype/development architecture.

2. Functional Requirements & Quality Expectations

ID	Requirement	Expected behavior
FR-01	Load knowledge	Read UTF-8 Markdown files from four domain folders.
FR-02	Preserve provenance	Retain source path and add `doc_type`.
FR-03	Create chunks	Split Documents with 1000 size / 200 overlap.
FR-04	Embed chunks	Use all-MiniLM-L6-v2 to create 384-D vectors.
FR-05	Persist index	Store text, metadata and vectors in `vector_db/`.
FR-06	Semantic search	Embed query with compatible model and search Chroma.
FR-07	Return evidence	Reusable runtime requests top 10 Documents.
FR-08	Conversational search	Use prior user turns in retrieval query.
FR-09	Ground prompt	Insert retrieved text into SYSTEM_PROMPT.
FR-10	Generate answer	Invoke selected model through OpenRouter.
FR-11	Return diagnostics	Return answer plus retrieved Documents.
FR-12	Evaluate quality	Run retrieval metrics and LLM answer evaluation.

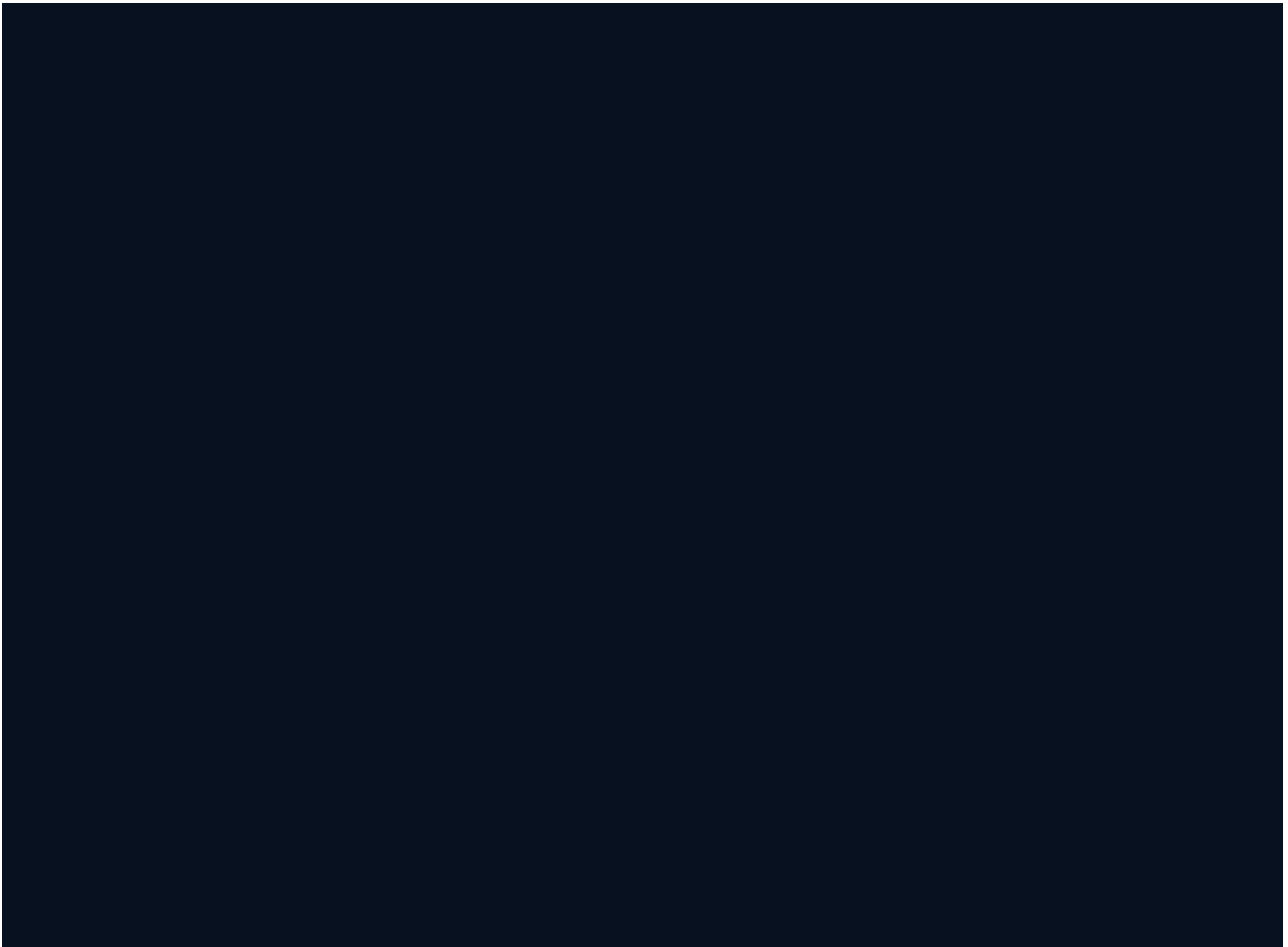
2.1 Quality dimensions

Dimension	How project measures / reasons about it
Retrieval rank	MRR and nDCG.
Evidence coverage	Expected-keyword coverage.
Factual accuracy	LLM-as-a-judge accuracy score.
Completeness	LLM judge completeness score.
Relevance	LLM judge relevance score.
Reproducibility	Restart kernel and rerun indexing/runtime from clean state.
Traceability	Keep retrieved Documents available with the answer.

3. Architecture — Canonical & Architect Views

3.1 Exact user-supplied `insurllm architecture.png`

The latest uploaded PNG is embedded below exactly as received, as explicitly requested.



Verification result: the uploaded PNG is near-uniform dark pixels in this environment, so no visible diagram content is present. The file is nevertheless embedded exactly. To avoid leaving the architecture section unusable, the next figure reconstructs the actual architecture from the codebase.

3.2 Code-grounded canonical architecture

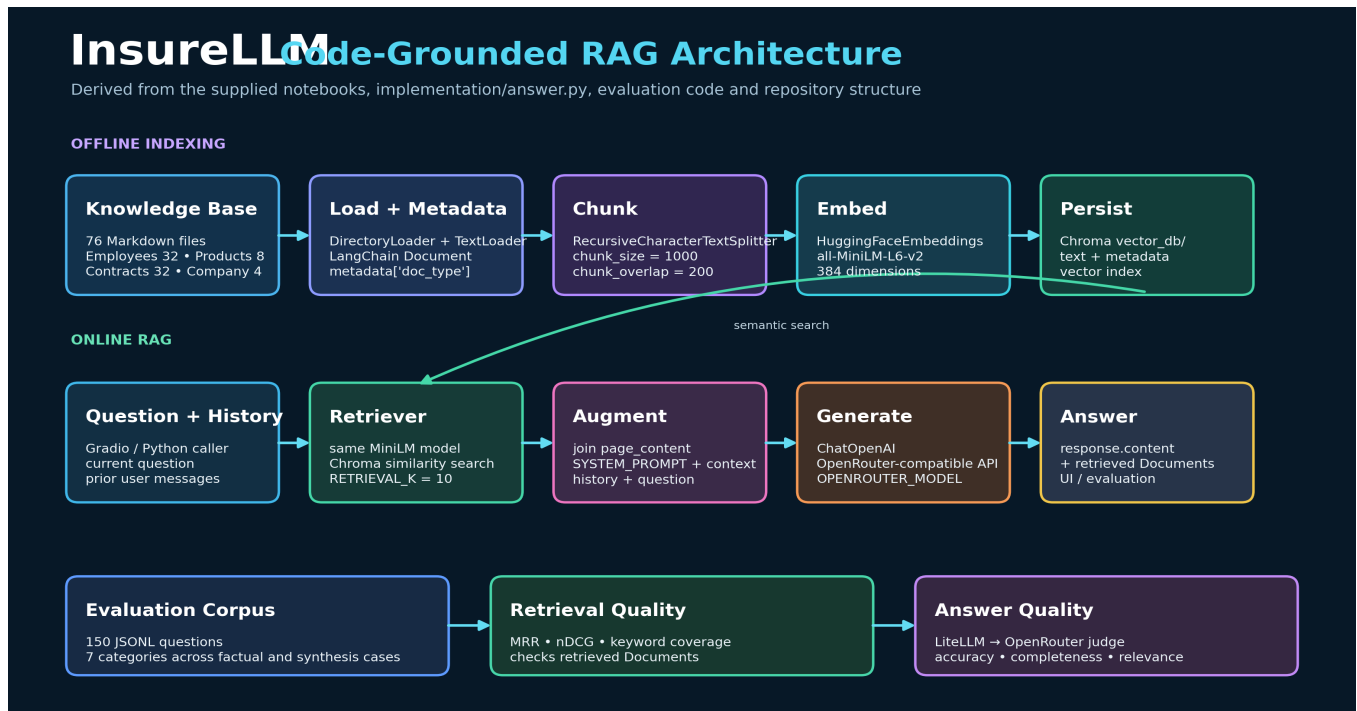


Figure 3.2 – Baseline InsureLLM architecture derived from the supplied code.

3.3 System context

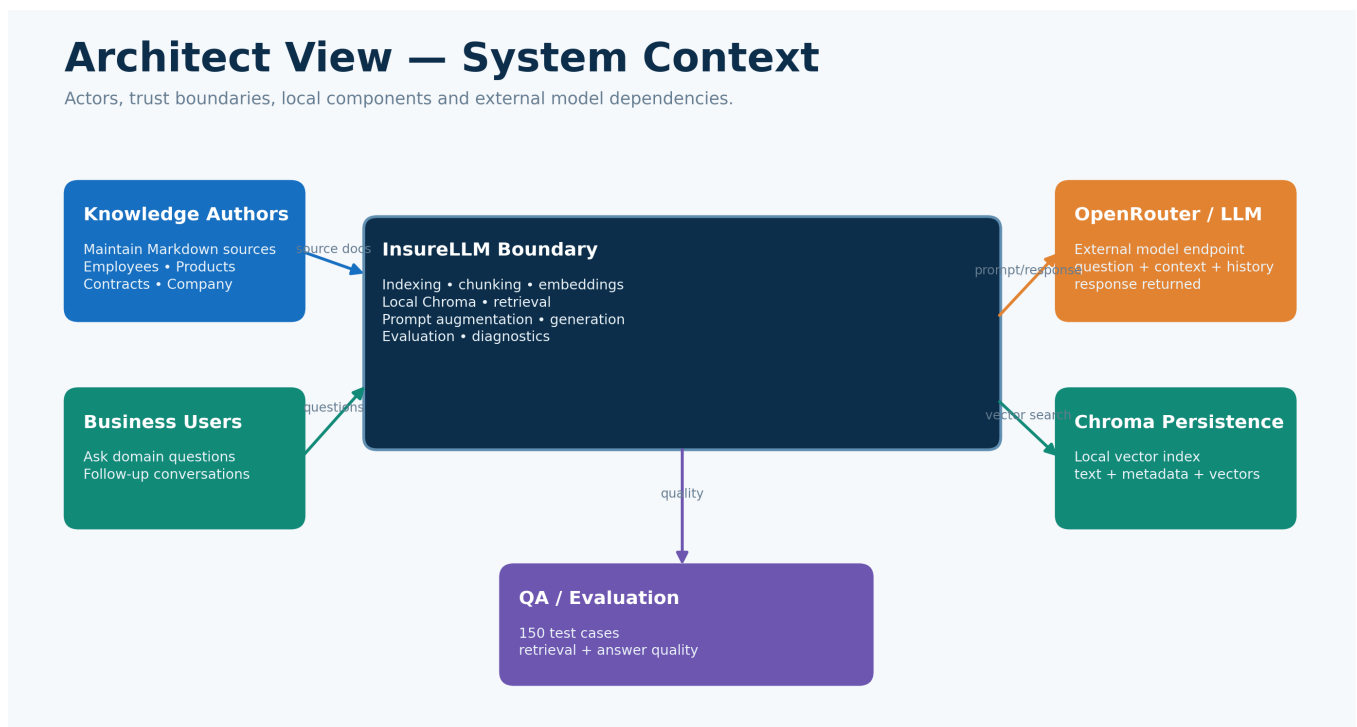


Figure 3.3 – Actors, system boundary and external trust boundaries.

3.4 Component architecture

Architect View — Component Architecture

Responsibilities, interfaces and coupling in the baseline RAG implementation.

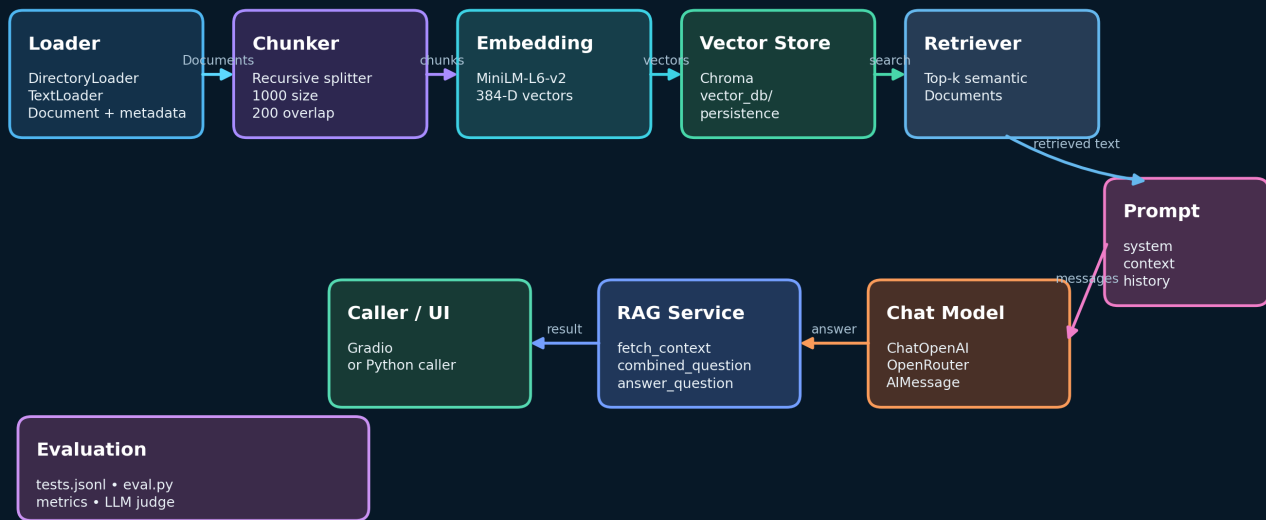
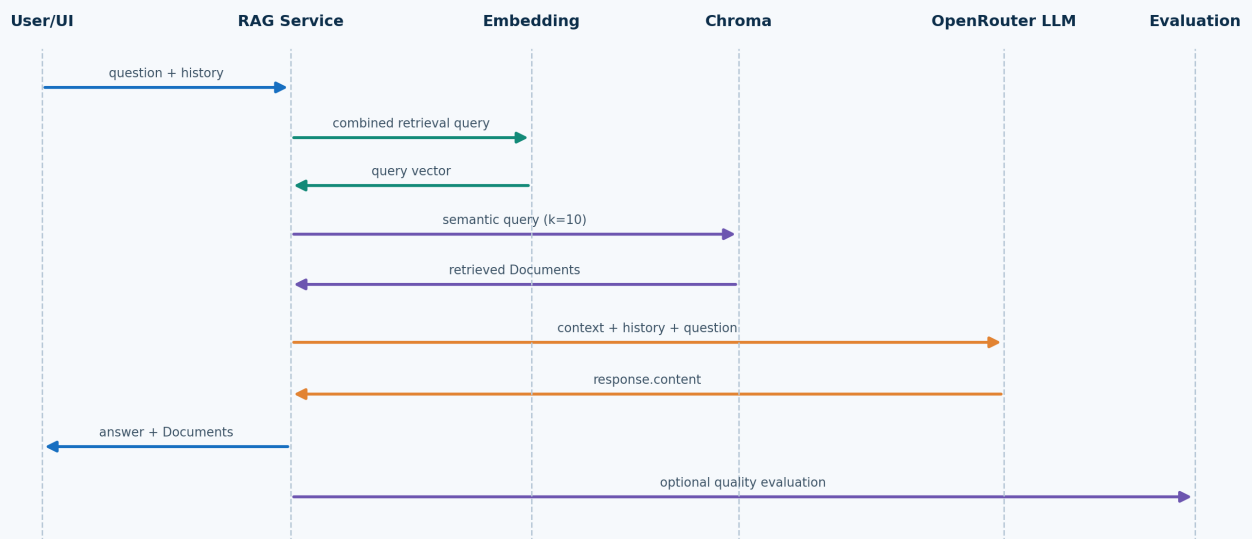


Figure 3.4 — Logical responsibilities and interfaces.

3.5 Runtime sequence

Architect View — Online RAG Sequence

Runtime ordering shows what happens before the LLM and where failures belong.



Debugging principle: inspect retrieved evidence first. Missing evidence = retrieval/index issue; good evidence + bad answer = generation issue.

Figure 3.5 — Online request sequence.

3.6 Deployment and production evolution

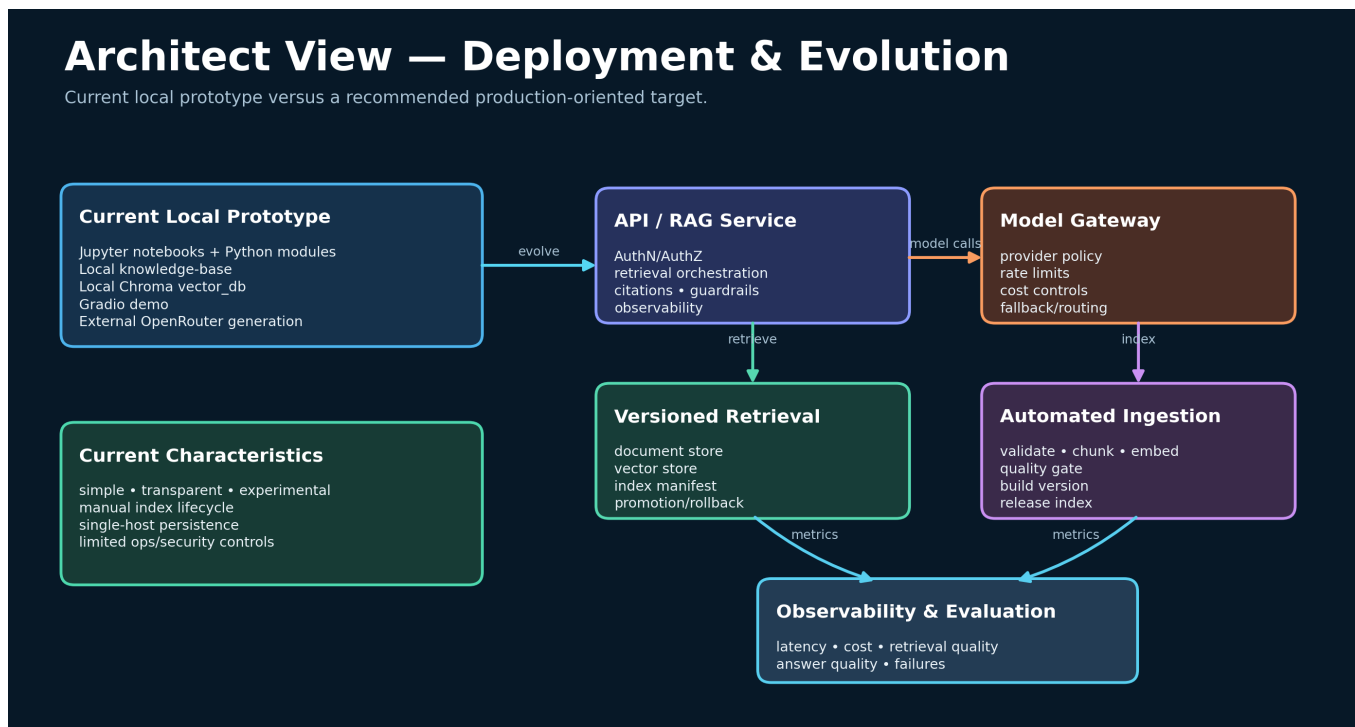


Figure 3.6 — Current prototype vs recommended production shape.

3.7 Architect interpretation

- Indexing and online retrieval are separate lifecycles.
- An index and the embedding model that created it form one compatibility unit.
- The LLM is downstream of retrieval; it does not directly query Chroma in the baseline implementation.
- Poor retrieval and poor generation are different failure domains and should have separate telemetry.
- External generation is a data-governance trust boundary because retrieved company context is sent to the provider.
- Production evolution requires identity, authorization, versioned indexes, observability and automated ingestion.

4. Technology Stack & Libraries

Library / technology	Role	Use in InsureLLM
Python	Core language	Notebooks, runtime, evaluation, scraper.
LangChain	Integration layer	Documents, loaders, splitters, messages, retriever, model adapters.
langchain-openai	Chat adapter	`ChatOpenAI` via OpenRouter-compatible endpoint.
langchain-chroma	Vector integration	Create/reopen/search Chroma.
langchain-huggingface	Embedding adapter	MiniLM embeddings.
langchain-core	Core contracts	Document and message objects.
langchain-community	Loaders	DirectoryLoader / TextLoader.
langchain-text-splitters	Chunking	RecursiveCharacterTextSplitter.
ChromaDB	Vector database	Persistent text/metadata/vector search.
Hugging Face	Embedding runtime	all-MiniLM-L6-v2.
OpenRouter	LLM gateway	Generation model access.
Gradio	UI	ChatInterface demo.
Pydantic	Schemas	Evaluation and advanced structured output.
LiteLLM	Model gateway	LLM judge and day5 structured calls.
tiktoken	Analysis	Token counting.
NumPy	Vector analysis	Embedding arrays.
scikit-learn	Visualization	t-SNE.
Plotly	Visualization	Interactive vector plots.
python-dotenv	Configuration	Loads local env vars.
requests + BeautifulSoup	Optional scraper	Fetch/clean website text and links.

4.1 LangChain's role

LangChain is not the model and not the vector database. It is the integration/orchestration layer that gives the project consistent interfaces for documents, embeddings, retrievers, messages and model invocation.

5. Developer Deep Dive — Code Snippet First, Explanation Second

Every technical subsection in this chapter shows the relevant code first and then explains it line by line or concept by concept. This is intentional and follows the learning style requested.

5.1 Core imports

CODE SNIPPET

```
from langchain_openai import ChatOpenAI
from langchain_chroma import Chroma
from langchain_core.messages import SystemMessage, HumanMessage
from langchain_huggingface import HuggingFaceEmbeddings
```

`ChatOpenAI` is the chat-model interface. `Chroma` connects LangChain to the vector database. `SystemMessage` and `HumanMessage` model chat roles. `HuggingFaceEmbeddings` wraps the active local embedding model.

5.2 Embedding model

CODE SNIPPET

```
embeddings = HuggingFaceEmbeddings(
    model_name="all-MiniLM-L6-v2"
)
```

MiniLM converts text into 384-dimensional vectors. The same embedding space must be used at index time and query time.

5.3 Build / rebuild Chroma

CODE SNIPPET

```
if os.path.exists(db_name):
    Chroma(
        persist_directory=db_name,
        embedding_function=embeddings
    ).delete_collection()

vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory=db_name
)
```

The notebook deletes the existing collection when rebuilding. `from\_documents()` embeds each chunk, then stores vector + text + metadata in the persistent Chroma location.

5.4 Retriever and LLM setup

CODE SNIPPET

```
retriever = vectorstore.as_retriever()

llm = ChatOpenAI(
    model="openai/gpt-4o-mini",
    api_key=api_key,
    base_url="https://openrouter.ai/api/v1",
    default_headers={
        "HTTP-Referer": "http://localhost",
        "X-Title": "llm_engineering"
    }
)

llm.invoke("hello")
```

`as\_retriever()` creates a retrieval interface; it finds Documents but does not generate an answer.

`ChatOpenAI(...)` is configured to send OpenAI-compatible chat calls to OpenRouter. The model field is an OpenRouter model identifier, the key authenticates the request, and the headers identify the local application.

`llm.invoke('hello')` invokes the model directly. Chroma is not searched during this call, so this line alone is not RAG.

5.5 Inspect embeddings and metadata

CODE SNIPPET

```
result = collection.get(
    include=['embeddings', 'documents', 'metadatas']
)

vectors = np.array(result['embeddings'])
documents = result['documents']
metadatas = result['metadatas']

doc_types = [
    metadata['doc_type']
    for metadata in metadatas
]

colors = [
    ['blue', 'green', 'red', 'orange'][
        ['products', 'employees', 'contracts', 'company'].index(t)
    ]
    for t in doc_types
]
```

Chroma returns embeddings, text and metadata in aligned positions: index `i` across all three corresponds to one stored chunk.

NumPy turns the embedding collection into a matrix; with MiniLM, each row has 384 values.

`doc\_types` extracts the category used for visualization. The final list comprehension chooses a color according to that category.

5.6 Custom `Result` model

CODE SNIPPET

```
class Result(BaseModel):
    page_content: str
    metadata: dict
```

This Pydantic model is a lightweight equivalent of the essential LangChain `Document` idea: retrieval text plus metadata/provenance.

5.7 Advanced `Chunk` and `Chunks` models

CODE SNIPPET

```
class Chunk(BaseModel):
    headline: str = Field(
        description="A brief heading for this chunk..."
    )

    summary: str = Field(
        description="A few sentences summarizing the content..."
    )

    original_text: str = Field(
        description="The original text exactly as is..."
    )

    def as_result(self, document):
        metadata = {
            "source": document["source"],
            "type": document["type"]
        }

        return Result(
            page_content=(
                self.headline + "\n\n"
                + self.summary + "\n\n"
                + self.original_text
            ),
            metadata=metadata
        )

class Chunks(BaseModel):
    chunks: list[Chunk]
```

`headline` and `summary` enrich the retrieval representation with likely query language and concise meaning. `original\_text` remains exact source evidence.

`Field(description=...)` helps a structured-output LLM understand what each field should contain.

`as\_result()` creates one embedding/retrieval string from headline + summary + source text and carries source/type in metadata.

`Chunks` is a typed list container used as a structured response format.

5.8 Reopen the persistent vector store

CODE SNIPPET

```
embeddings = HuggingFaceEmbeddings(  
    model_name="all-MiniLM-L6-v2"  
)  
  
vectorstore = Chroma(  
    persist_directory=DB_NAME,  
    embedding_function=embeddings  
)  
  
RETRIEVAL_K = 10  
  
retriever = vectorstore.as_retriever(  
    search_kwargs={"k": RETRIEVAL_K}  
)
```

The runtime reopens an existing index rather than rebuilding it. The query embedding function must remain compatible with the stored vectors. The reusable implementation explicitly requests the top 10 Documents.

5.9 `fetch\_context()`

CODE SNIPPET

```
def fetch_context(  
    question: str  
) -> list[Document]:  
    return retriever.invoke(question)
```

This is the clean retrieval boundary: text in, relevant Documents out. It is an excellent place to add tracing, timing and unit/integration tests.

5.10 Conversation-aware retrieval query

CODE SNIPPET

```
def combined_question(question, history):  
    user_messages = [  
        item["content"]  
        for item in history  
        if item["role"] == "user"  
    ]  
  
    user_messages.append(question)  
  
    return "\n".join(user_messages)
```

Earlier user turns are concatenated with the current question. This gives follow-up questions more semantic context without using a separate rewrite model in the baseline.

5.11 Complete RAG answer function

CODE SNIPPET

```
def answer_question(question, history=[]):
    combined = combined_question(
        question,
        history
    )

    docs = fetch_context(combined)

    context = "\n\n".join(
        doc.page_content
        for doc in docs
    )

    system_prompt = SYSTEM_PROMPT.format(
        context=context
    )

    messages = [
        SystemMessage(
            content=system_prompt
        )
    ]

    messages.extend(
        convert_to_messages(history)
    )

    messages.append(
        HumanMessage(
            content=question
        )
    )

    response = llm.invoke(messages)

    return response.content, docs
```

First retrieve; then augment; then generate. The final tuple deliberately includes both the model answer and the retrieved Documents, which is valuable for evaluation, debugging and future citations.

5.12 Embedding visualization

CODE SNIPPET

```
vectors = np.array(
    result['embeddings']
)

tsne = TSNE(
    n_components=2,
    random_state=42
)

reduced = tsne.fit_transform(
    vectors
)

fig = go.Figure(
    data=[
        go.Scatter(
            x=reduced[:, 0],
            y=reduced[:, 1],
            mode='markers',
            marker=dict(
                color=colors
            )
        )
    ]
)

fig.show()
```

t-SNE compresses high-dimensional vectors into 2-D for exploratory visualization. The x/y axes are not original embedding dimensions and should not be interpreted as actual Chroma similarity scores.

5.13 Restarting the Jupyter kernel

CODE SNIPPET

```
# Jupyter / JupyterLab.
# Kernel -> Restart Kernel
# Then rerun cells from the top.
```

A kernel restart clears variables and in-memory objects. For InsureLLM notebooks, restarting and running top-to-bottom is a useful reproducibility check and exposes hidden dependencies on stale notebook state.

6. QA & Evaluation

6.1 Evaluation corpus

Category	Count	Purpose
direct_fact	70	Single-source factual retrieval.
spanning	20	Multiple passages.
temporal	20	Dates/time/order.
comparative	10	Entity comparison.
holistic	10	Broader synthesis.
numerical	10	Numbers/counts.
relationship	10	Entity relationships.

6.2 Retrieval metrics

- MRR: rewards expected evidence appearing early.
- nDCG: measures relevance with rank discounting.
- Keyword coverage: checks whether expected evidence is present in retrieved chunks.

6.3 Answer metrics

The generated answer is judged against the reference answer through LiteLLM/OpenRouter. Pydantic structures the result as accuracy, completeness, relevance and feedback.

6.4 QA scenario matrix

Area	Positive	Negative / edge
Index	Fresh DB contains all domains.	Missing/stale DB or incompatible embedding model.
Retrieval	Known question retrieves evidence.	Typo, synonym, vague query, unknown entity.
Conversation	Follow-up carries prior context.	Long/noisy history changes retrieval.
Generation	Answer agrees with evidence.	Good evidence but incorrect/hallucinated answer.
Provider	Valid key/model works.	Invalid key, unavailable model, timeout.
Evaluation	Valid fixture returns scores.	Malformed JSONL, judge failure.
UI	Question renders response.	Empty/long input, repeated error.

6.5 Regression gate

- Establish baseline metrics before a RAG change.
- Evaluate retrieval changes and generation changes separately.
- Review results per category, not only overall average.
- Inspect worst examples manually.
- Track latency and cost together with quality.

7. Architect Deep Dive

7.1 Architectural quality attributes

Attribute	Current baseline	Architect action
Simplicity	Local notebook/module architecture.	Preserve clarity while extracting production services.
Modularity	Indexing/runtime/evaluation conceptually separate.	Formalize module/service boundaries.
Explainability	Retrieved Documents returned internally.	Expose source/citation path.
Scalability	Local single-host Chroma.	Choose retrieval topology based on corpus and QPS.
Security	Env secrets; no app auth model.	AuthN/AuthZ + document policy + provider governance.
Reliability	No full HA/fallback strategy.	Timeouts, retries, provider fallback, health signals.
Observability	Notebook inspection and evaluation.	Trace query, result IDs/scores, latency, tokens, cost.
Index lifecycle	Notebook deletes/rebuilds current collection.	Versioned build, evaluate, promote, rollback.

7.2 Recommended index identity

Version source corpus + chunking configuration + embedding model/version + metadata schema together. A change to any member should produce a new candidate index rather than silently modifying the active one.

7.3 Production service decomposition

- Ingestion service: validates sources, chunks, embeds and builds a candidate index.
- Retrieval service: owns search, filters, thresholds and authorization-aware access.
- RAG service: owns conversation/query handling, prompt construction and generation.
- Model gateway: owns provider routing, timeouts, retry, cost and fallback.
- Evaluation/observability plane: tracks retrieval quality, answer quality, performance and spend.

7.4 Trust boundaries

- Source documents may contain sensitive organizational knowledge.
- Chroma persistence is local in the current project.
- Baseline embeddings can run locally after model availability.
- Question + history + retrieved context can leave the local process during external LLM generation.
- LLM-based evaluation can also send generated/reference content externally.
- Data classification and provider policy are architecture requirements.

8. Security, Privacy & Operations

8.1 Secret handling

RECOMMENDED SOURCE-CONTROL EXCLUSIONS

```
.env
.env.*
__pycache__/
*.pyc
.ipynb_checkpoints/
```

Real API keys must not be embedded in notebooks, screenshots, committed source, documentation or logs.

8.2 RAG-specific risk areas

- Prompt injection inside ingested documents.
- Unauthorized retrieval of restricted content.
- Knowledge poisoning through unreviewed source updates.
- Stale indexes producing stale answers.
- Hallucinations when retrieved evidence is weak.
- Sensitive context leakage in logs, traces or evaluation artifacts.

8.3 Production telemetry

Signal	Purpose
Retrieval latency	Search performance.
LLM latency	Provider/model performance.
Token usage	Cost/context pressure.
Result rank/score	Evidence diagnosis.
Unknown/abstention rate	Safe uncertainty.
Provider error rate	Reliability/fallback need.
Index version	Reproducibility.
Evaluation trend	Quality regression detection.

9. Engineering Manager / Product View

9.1 Business value

- Natural-language access to private organization knowledge.
- Reduced manual search across internal documents.
- Measurable quality rather than demo-only success.
- Model flexibility without retraining the source knowledge.
- Foundation for governed assistants with citations and access controls.

9.2 Roadmap priorities

Priority	Outcome
P0	Pinned dependencies, reproducible setup, <code>.env.example</code> , automated smoke tests.
P1 – Trust	Source citations, stronger grounding, threshold/abstention.
P1 – Retrieval	Metadata filters; evaluate hybrid search/reranking before adoption.
P1 – Operations	Move ingestion/runtime out of notebooks into tested services/modules.
P2 – Governance	AuthN/AuthZ, auditing, index versioning.
P2 – Observability	Quality, latency, cost, provider health dashboards.

10. Advanced RAG — `day5.ipynb` (Placed Last Intentionally)

The advanced notebook should be read only after the baseline RAG is understood. It changes chunking, embeddings, retrieval orchestration and ranking, and therefore represents a separate architectural experiment.

10.1 Configuration

CODE SNIPPET

```
MODEL = "gpt-4.1-mini"
DB_NAME = "preprocessed_db"
collection_name = "docs"
embedding_model = "text-embedding-3-large"
KNOWLEDGE_BASE_PATH = Path("knowledge-base")
AVERAGE_CHUNK_SIZE = 500
```

The advanced pipeline creates a separate index (`preprocessed\_db`) and uses a different embedding model. Baseline and advanced indexes are not interchangeable.

10.2 Native document loading

CODE SNIPPET

```
def fetch_documents():
    documents = []

    for folder in KNOWLEDGE_BASE_PATH.iterdir():
        doc_type = folder.name

        for file in folder.rglob("*.md"):
            with open(file, "r", encoding="utf-8") as f:
                documents.append({
                    "type": doc_type,
                    "source": file.as_posix(),
                    "text": f.read()
                })

    return documents
```

This replaces LangChain's document loader with `pathlib` and plain dictionaries for maximum control.

10.3 LLM semantic chunking

CODE SNIPPET

```
def process_document(document):
    messages = make_messages(document)

    response = completion(
        model=MODEL,
        messages=messages,
        response_format=Chunks
    )

    reply = response.choices[0].message.content

    doc_as_chunks = Chunks.model_validate_json(
        reply
    ).chunks

    return [
        chunk.as_result(document)
        for chunk in doc_as_chunks
    ]
```

An LLM chooses semantic chunk boundaries and returns the structured `Chunks` schema. This can improve chunk meaning but adds preprocessing cost, latency, rate-limit exposure and model-dependent failure modes.

10.4 Advanced embedding and native Chroma

CODE SNIPPET

```
texts = [
    chunk.page_content
    for chunk in chunks
]

emb = openai.embeddings.create(
    model=embedding_model,
    input=texts
).data

vectors = [
    item.embedding
    for item in emb
]

collection.add(
    ids=ids,
    embeddings=vectors,
    documents=texts,
    metadatas=metas
)
```

The enriched chunk representation is embedded with `text-embedding-3-large`; vectors are supplied directly to native Chroma.

10.5 Query rewrite

CODE SNIPPET

```
def rewrite_query(question, history=[]):
    response = completion(
        model=MODEL,
        messages=[
            {
                "role": "system",
                "content": message
            }
        ]
    )

    return response.choices[0].message.content
```

The LLM converts conversational input into a concise standalone knowledge-base query. This is especially useful for pronoun-heavy follow-up questions.

10.6 Candidate retrieval

CODE SNIPPET

```
query = openai.Embeddings.create(
    model=embedding_model,
    input=[question]
).data[0].embedding

results = collection.query(
    query_embeddings=[query],
    n_results=RETRIEVAL_K
)
```

The rewritten query is embedded in the same advanced vector space and Chroma returns candidate results.

10.7 LLM reranking

CODE SNIPPET

```
class RankOrder(BaseModel):
    order: list[int] = Field(
        description="The order of relevance..."
    )

order = RankOrder.model_validate_json(
    response.choices[0].message.content
).order

reranked = [
    chunks[i - 1]
    for i in order
]
```

The reranker receives vector-search candidates and returns a 1-based relevance order. `i - 1` converts that order to Python list indexes.

10.8 Final advanced answer function

CODE SNIPPET

```
def answer_question(question, history=[]):
    query = rewrite_query(
        question,
        history
    )

    chunks = fetch_context(
        query
    )

    messages = make_rag_messages(
        question,
        history,
        chunks
    )

    response = completion(
        model=MODEL,
        messages=messages
    )

    return (
        response.choices[0].message.content,
        chunks
    )
```

The advanced online flow is rewrite → vector retrieval → rerank → source-aware prompt → generation. Offline, LLM semantic chunking happens before indexing.

10.9 Baseline vs advanced

Dimension	Baseline	Advanced
Chunking	Deterministic 1000/200.	LLM semantic chunking + enrichment.
Embedding	Local MiniLM, 384-D.	`text-embedding-3-large`.
Query handling	Concatenate prior user turns.	LLM standalone-query rewrite.
Ranking	Vector similarity.	Vector candidates + LLM rerank.
Latency	Retrieval + generation.	Rewrite + retrieval + rerank + generation.
Cost	Mostly generation calls.	Preprocess + embeddings + rewrite + rerank + answer.
Complexity	Lower.	Higher: more external stages and schemas.

Recommendation: A/B test baseline and advanced pipelines against the same 150-question evaluation corpus. Adopt the advanced stages only when measurable quality improvements justify extra latency, cost, data egress and operational complexity.

11. Onboarding Checklist & Mental Models

11.1 New developer / QA sequence

- Read functional Sections 1–2 before looking at implementation.
- Study the canonical architecture and Architect views in Section 3.
- Read Section 5 in order: every code snippet precedes its explanation.
- Run Part 2 indexing and inspect Chroma content.
- Read `implementation/answer.py` end-to-end and inspect retrieved Documents.
- Review the 150-question evaluation suite.
- Study architecture/security before production changes.
- Read day5 only after baseline RAG is clear.

11.2 Baseline mental model

Markdown → Document → chunks → MiniLM vectors → Chroma → top-10 retrieval → prompt context → OpenRouter LLM → answer + Documents → evaluation.

11.3 Advanced mental model

Document → LLM semantic chunks → enriched Result → OpenAI embeddings → native Chroma → LLM query rewrite → vector candidates → LLM rerank → source-aware prompt → LLM answer.

11.4 Definition of done for a RAG change

- Rebuild/version index if corpus, chunking or embedding changed.
- Run smoke questions across all knowledge domains.
- Compare retrieval metrics with baseline.
- Review answer-quality regressions by category.
- Measure latency/cost.
- Verify security/data-governance implications.
- Document the change and rollback path.